

Day 2 - HTML Forms, Tables, and Semantic HTML5

Day 2 — HTML Forms, Tables, and Semantic HTML5

Objectives

- Build a real HTML form with several input types, and understand the connection between a form and the data it would send somewhere
- Understand why `<label>` matters for accessibility, not just appearance
- Build a table for genuinely tabular data, and know when NOT to reach for one
- Meet HTML5's semantic layout elements and understand why they exist instead of just using `<div>` everywhere

Forms — collecting input from a visitor

A **form** is how a web page collects information from someone visiting it — a login screen, a search box, a contact form, a checkout page. The wrapping element is `<form>`:

```
<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">

  <button type="submit">Submit</button>
</form>
```

`action` says where the form's data should be sent when submitted (a web address on a server — you won't have a real server to send this to in this HTML/CSS-only course, so don't worry about `action` actually working yet; it becomes relevant once you're pairing this with a backend, which is outside this track's scope). `method` says how the data is sent ("`get`" or "`post`" — a distinction that matters once you're actually building a server to receive it, again outside today's scope). For today, you're focused on correctly *building* the form's structure — connecting it to a real server is a separate skill for later.

`<label>` and `for`/`id` — a genuinely important accessibility connection

```
<label for="email">Email address:</label>
<input type="email" id="email" name="email">
```

The `for` attribute on `<label>` must exactly match the `id` attribute on the input it describes. This creates a real, functional connection — not just a visual one — between the label text and its input: clicking the label text itself moves focus into (or, for a checkbox, toggles) the associated input, and screen readers announce the label's text whenever that input receives focus. **Never skip `<label>`, and never rely on placeholder text alone as a substitute** — placeholder text (shown below) disappears the instant someone starts typing, and isn't reliably announced by every screen reader the same way a proper `<label>` is.

Common `<input>` types

```








```

`type` changes both how the input is displayed (a password field hides typed characters; a number field often shows small up/down arrows) and, on many modern browsers and mobile devices, what kind of keyboard or built-in validation is offered (an `email` field can show an @-friendly keyboard on a phone, and browsers can reject obviously malformed input automatically — much more on this validation behavior on Day 12). Radio buttons that should behave as one connected group (only one selectable at a time) must all share the exact same `name` attribute, as shown above with `"size"` — this is how the browser knows they belong together and enforces "only one selected."

Other common form elements:

```

<textarea name="message" rows="4" cols="40">Default text here</textarea>

<select name="country">
  <option value="us">United States</option>
  <option value="ca">Canada</option>
  <option value="uk">United Kingdom</option>
</select>

<button type="submit">Send Message</button>

```

`<textarea>` is for multi-line free text (a comment box, a message). `<select>` with nested `<option>` elements creates a dropdown menu. `<button type="submit">` (or `<input type="submit">`) submits the form when clicked — note the difference between `type="submit"` (submits the form) and `type="button"` (does nothing on its own; used when you'll later attach custom JavaScript behavior to it, outside this course's scope).

Tables — for genuinely tabular data

```

<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>City</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ana</td>
      <td>30</td>
      <td>Boston</td>
    </tr>
    <tr>
      <td>Bo</td>
      <td>25</td>
      <td>Chicago</td>
    </tr>
  </tbody>
</table>

```

```
        </tr>
      </tbody>
    </table>
```

`<table>` wraps the whole table. `<thead>` groups the header row(s); `<tbody>` groups the actual data rows (both are optional but good practice, and genuinely useful once you meet CSS styling and want to target headers differently from data rows). `<tr>` is one table row ("table row"). Inside a row, `<th>` ("table header") marks a header cell (browsers bold and center it by default), while `<td>` ("table data") marks an ordinary data cell.

A genuinely important rule, worth stating plainly: only use `<table>` for data that is actually tabular — rows and columns of related data, like a spreadsheet, a price list, or a schedule. **Do not use a `<table>` purely to arrange unrelated content visually on the page** (like a page layout with a "sidebar column" and a "main content column") — this was common practice in the early days of the web, is now considered outdated and actively harmful to accessibility (screen readers announce table structure — row and column counts, headers — which is confusing and meaningless when the "table" was never really tabular data at all), and has been fully superseded by CSS layout tools you'll learn Day 5 (Flexbox) and Day 6 (Grid), which are the correct tools for arranging a page's visual layout.

Semantic HTML5 — elements that describe a page's actual layout regions

You've been using `<div>` and `<span>` implicitly so far only through prose descriptions — these are **generic, meaningless containers**: a `<div>` is a generic block-level box, and a `<span>` is a generic inline box, and neither says anything at all about what the content inside actually *is*. HTML5 introduced a set of elements that describe common, recognizable *regions* of a typical page layout, carrying real semantic meaning:

```
<body>
  <header>
    <h1>My Site</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
      <a href="/contact">Contact</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>My First Blog Post</h2>
      <p>Content of the post goes here...</p>
    </article>

    <aside>
      <h3>Related Links</h3>
      <ul><li><a href="#">Another post</a></li></ul>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 My Site</p>
  </footer>
</body>
```

- `<header>` — introductory content for the page or a section (often contains a logo/title and navigation, but can appear more than once, e.g., inside an `<article>` for that article's own header).
- `<nav>` — a block of primary navigation links.
- `<main>` — the single, primary, unique content of the page (use exactly one per page).
- `<article>` — a self-contained, independently distributable piece of content — a blog post, a news story, a forum post — content that would still make complete sense on its own if pulled out and placed somewhere else entirely.
- `<section>` — a thematic grouping of content, generally with its own heading, when there isn't a more specific element (like `<article>`) that fits better.
- `<aside>` — content tangentially related to the surrounding content, like a sidebar, a pull-quote, or related links.
- `<footer>` — closing content for the page or a section (copyright notices, contact info, secondary links).

Why bother with these instead of just using `<div>` for everything, especially since, by default, many of these look identical to a `<div>` until you apply CSS? The exact same reasoning from Day 1's heading discussion applies here, at the scale of a whole page: screen readers can let a visitor jump directly to "the navigation" or "the main content" without listening to the entire page read aloud first; search engines use this structure to better understand which part of your page is the actual content versus boilerplate surrounding it; and another developer (including future-you) can understand your page's layout at a glance from the tag names alone, without needing to guess what a generically-named `<div class="thing1">` was supposed to represent.

A practical rule of thumb: reach for the semantic element whenever one genuinely describes the region you're building (header, nav, main, article, section, aside, footer). Fall back to a plain `<div>` only when you need a container purely for layout or styling purposes, with no real semantic meaning of its own — you'll use both throughout the rest of this course, and knowing which to reach for is itself a skill you're building starting today.

Exercises

Open `starter.html`, follow the `<!-- TODO -->` markers, and check your work against `CHECKLIST.md`.